04 - Etapas: Implementación de MCP Blender

Resumen de Etapas

```
Etapa 1: Setup e Infraestructura
  ├─ Crear proyecto Node.js/TypeScript
  ├─ Instalar Blender CLI
  ├─ Crear estructura de scripts Python

Etapa 2: Herramientas Básicas de Modelado (Fase I)
  ├─ CreateModel (mesh humanoide básico)
  ├─ EditModel (modificar propiedades)
  └─ DeleteModel (eliminar modelo)

Etapa 3: Herramientas de Rigging (Fase II)
  ├─ AddRig (agregar armadura estándar)
  ├─ CreateArmature (crear bones manuales)
  └─ ApplyRigify (automatizar rigging)

Etapa 4: Herramientas de Animación (Fase III)
  ├─ CreateAnimation
  ├─ EditKeyframes
  └─ PlayAnimation (preview)

Etapa 5: Exportación (Fase IV)
  ├─ ExportFBX
  ├─ ExportGLB
  ├─ OptimizeModel
  └─ BatchExport
```

ETAPA 1: Setup e Infraestructura

1.1 Estructura de carpetas

```
# En MCP/servers/blender/
mkdir -p src/{tools,utils} build python

# Crear archivos base
touch package.json tsconfig.json src/index.ts
touch src/blender-cli.ts src/types.ts

# Scripts Python para Blender
touch python/create_model.py
touch python/add_rig.py
touch python/create_animation.py
```

```
touch python/export_fbx.py
touch python/utils.py
```

1.2 Verificar instalación de Blender

```
# Verificar que Blender existe
which blender
# o en macOS
/Applications/Blender/blender --version

# Verificar Python en Blender
blender --background --python-exit-code 1 --python - <<EOF
import sys
print(f"Python {sys.version}")
EOF
```

1.3 package.json

```
{
  "name": "zoremgame-mcp-blender",
  "version": "1.0.0",
  "description": "MCP Server for Blender 3D modeling automation",
  "main": "build/index.js",
  "scripts": {
    "build": "tsc",
    "dev": "ts-node src/index.ts",
    "start": "node build/index.js"
  },
  "dependencies": {
    "@modelcontextprotocol/sdk": "latest",
    "jsonrpc-lite": "^2.3.0",
    "child_process": "latest",
    "fs-extra": "^11.0.0",
    "uuid": "^9.0.0"
  },
  "devDependencies": {
    "@types/node": "^20.0.0",
    "typescript": "^5.0.0",
    "ts-node": "^10.0.0"
  }
}
```

1.4 tsconfig.json

```
{
  "compilerOptions": {
```

```
    "target": "ES2020",
    "module": "commonjs",
    "lib": ["ES2020"],
    "outDir": "./build",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules"]
}
```

1.5 src/types.ts

```
export interface BlenderProjectConfig {
  blenderExecutable: string;
  outputPath: string;
  pythonScriptsPath: string;
  pythonVersion: string;
  defaultFormat: "fbx" | "glb" | "usdz";
}

export interface ModelSpec {
  type: "humanoid" | "cube" | "sphere" | "custom";
  height?: number;
  width?: number;
  depth?: number;
  rigged?: boolean;
}

export interface ExportOptions {
  format: "fbx" | "glb" | "usdz";
  applyModifiers?: boolean;
  autoSmooth?: boolean;
  scale?: number;
  optimized?: boolean;
}

export interface AnimationSpec {
  name: string;
  frameStart: number;
  frameEnd: number;
  fps?: number;
}

export interface BlenderResult {
  success: boolean;
  message: string;
  filepath?: string;
```

```
  metadata?: Record<string, any>;
}
```

1.6 src/blender-cli.ts

```
import { exec, execSync } from "child_process";
import { promisify } from "util";
import * as fs from "fs-extra";
import * as path from "path";
import { v4 as uuidv4 } from "uuid";

const execAsync = promisify(exec);

export class BlenderCLI {
  private blenderExecutable: string;
  private pythonScriptsPath: string;
  private tempPath: string;

  constructor(
    blenderExecutable: string,
    pythonScriptsPath: string,
    tempPath: string = "/tmp"
  ) {
    this.blenderExecutable = blenderExecutable;
    this.pythonScriptsPath = pythonScriptsPath;
    this.tempPath = tempPath;
  }

  /**
   * Ejecuta un script Python dentro de Blender
   */
  async runPythonScript(
    scriptPath: string,
    args: Record<string, string> = {},
    timeout: number = 120000
  ): Promise<{ stdout: string; stderr: string }> {
    // Crear un wrapper que pase argumentos
    const argsJson = JSON.stringify(args);
    const wrapperScript = `
import sys
import json

# Obtener argumentos pasados
args = json.loads('${argsJson.replace(/'/g, "\\'")}')

# Ejecutar script original
with open('${scriptPath}', 'r') as f:
    exec(f.read())
`;

    const tempScript = path.join(this.tempPath, `blender_${uuidv4()}.py`);
```

```
    await fs.writeFile(tempScript, wrapperScript);

    try {
      const command = `"${this.blenderExecutable}" \
        --background \
        --python "${tempScript}" \
        --python-exit-code 1`;

      const result = await execAsync(command, {
        timeout,
        maxBuffer: 50 * 1024 * 1024,
      });

      return result;
    } finally {
      await fs.remove(tempScript);
    }
  }

  /**
   * Crea un proyecto Blender en blanco
   */
  async createBlendFile(
    outputPath: string,
    template: string = "empty"
  ): Promise<string> {
    const script = `
import bpy

# Limpiar scene default
bpy.ops.object.select_all(action='SELECT')
bpy.ops.object.delete()

# Salvar
bpy.ops.wm.save_as_mainfile(filepath='${outputPath}')
`;

    await this.runPythonScript("create_empty.py", { outputPath });
    return outputPath;
  }

  /**
   * Lista objetos en un archivo .blend
   */
  async listObjects(blendFilePath: string): Promise<string[]> {
    const script = `
import bpy
bpy.ops.wm.open_mainfile(filepath='${blendFilePath}')

for obj in bpy.data.objects:
    print(f"Object: {obj.name} ({obj.type})")
`;

    const result = await this.runPythonScript("list_objects.py");
```

```
    return result.stdout.split("\n").filter((line) => line.startsWith("Object:"));
  }

  /**
   * Exporta a FBX
   */
  async exportFBX(
    blendFilePath: string,
    outputPath: string,
    options: Record<string, any> = {}
  ): Promise<string> {
    const script = `
import bpy
import json

bpy.ops.wm.open_mainfile(filepath='${blendFilePath}')

# Aplicar modifiers si está requerido
if ${options.applyModifiers || false}:
    for obj in bpy.context.scene.objects:
        if obj.modifiers:
            bpy.context.view_layer.objects.active = obj
            for mod in obj.modifiers:
                bpy.ops.object.modifier_apply(modifier=mod.name)

# Exportar
bpy.ops.export_scene.fbx(
    filepath='${outputPath}',
    use_selection=False,
    object_types={'ARMATURE', 'MESH', 'EMPTY'},
    use_mesh_modifiers=True,
    scale_objects=True,
    axis_forward='-Y',
    axis_up='Z'
)

print(f"Exported to: {outputPath}")
`;

    await this.runPythonScript("export_fbx.py");
    return outputPath;
  }

  /**
   * Valida que Blender está instalado y funciona
   */
  async validateInstallation(): Promise<boolean> {
    try {
      const result = execSync(`"${this.blenderExecutable}" --version`).toString();
      return result.includes("Blender");
    } catch {
      return false;
    }
```

```
  }
}
```

1.7 src/index.ts (Entry Point - Blender)

```
import { Server } from "@modelcontextprotocol/sdk/server/index.js";
import { CallToolRequestSchema } from "@modelcontextprotocol/sdk/types.js";
import * as fs from "fs-extra";
import * as path from "path";
import { BlenderCLI } from "./blender-cli";
import { BlenderProjectConfig, ModelSpec, ExportOptions } from "./types";

const config: BlenderProjectConfig = {
  blenderExecutable:
    process.env.BLENDER_EXECUTABLE || "/Applications/Blender/blender",
  outputPath: process.env.OUTPUT_PATH || "/tmp/blender_exports",
  pythonScriptsPath: process.env.BLENDER_SCRIPTS || "./python",
  pythonVersion: process.env.BLENDER_PYTHON_VERSION || "3.10",
  defaultFormat: (process.env.EXPORT_FORMAT as "fbx" | "glb") || "fbx",
};

const cli = new BlenderCLI(
  config.blenderExecutable,
  config.pythonScriptsPath,
  "/tmp"
);
const server = new Server({
  name: "zoremgame-blender-mcp",
  version: "1.0.0",
});

// ==========================================
// HERRAMIENTAS (Tools)
// ==========================================

/**
 * Tool: CreateModel
 * Crea un modelo 3D básico en Blender
 */
async function handleCreateModel(input: {
  type: "humanoid" | "cube" | "sphere";
  name: string;
  height?: number;
}): Promise<any> {
  try {
    if (!input.name) throw new Error("name is required");

    const blendFile = path.join(config.outputPath, `${input.name}.blend`);
    await fs.ensureDir(config.outputPath);

    // Generar script según tipo
```

```
    let script = "";
    switch (input.type) {
      case "humanoid":
        script = `
import bpy
from mathutils import Vector

# Agregar armadura humana básica
bpy.ops.object.armature_add(
    name='${input.name}_Rig',
    enter_editmode=False
)

# Agregar mesh humanoid
bpy.ops.mesh.primitive_uv_sphere_add(radius=0.5, location=(0, 0, 1.7))
sphere = bpy.context.active_object
sphere.name = '${input.name}'

bpy.ops.wm.save_as_mainfile(filepath='${blendFile}')
print("Model created: ${blendFile}")
`;
        break;

      case "cube":
        script = `
import bpy
bpy.ops.mesh.primitive_cube_add(size=1)
obj = bpy.context.active_object
obj.name = '${input.name}'
bpy.ops.wm.save_as_mainfile(filepath='${blendFile}')
`;
        break;

      case "sphere":
        script = `
import bpy
bpy.ops.mesh.primitive_uv_sphere_add(radius=1)
obj = bpy.context.active_object
obj.name = '${input.name}'
bpy.ops.wm.save_as_mainfile(filepath='${blendFile}')
`;
        break;
    }

    // Ejecutar
    await cli.runPythonScript(path.join(config.pythonScriptsPath,
"create_model.py"));

    return {
      content: [
        {
          type: "text",
          text: `Model created: ${input.name}\nType: ${input.type}\nPath:
${blendFile}`,
```

```
        },
      ],
    };
  } catch (error) {
    throw new Error(`CreateModel failed: ${error}`);
  }
}

/**
 * Tool: ExportFBX
 * Exporta modelo a FBX
 */
async function handleExportFBX(input: {
  blend_file: string;
  output_path: string;
  apply_modifiers?: boolean;
}): Promise<any> {
  try {
    if (!input.blend_file || !input.output_path) {
      throw new Error("blend_file and output_path are required");
    }

    const outputDir = path.dirname(input.output_path);
    await fs.ensureDir(outputDir);

    await cli.exportFBX(input.blend_file, input.output_path, {
      applyModifiers: input.apply_modifiers || true,
    });

    return {
      content: [
        {
          type: "text",
          text: `Exported to FBX: ${input.output_path}`,
        },
      ],
    };
  } catch (error) {
    throw new Error(`ExportFBX failed: ${error}`);
  }
}

// ============================================
// SETUP DEL SERVER
// ============================================

server.setRequestHandler(CallToolRequestSchema, async (request) => {
  switch (request.params.name) {
    case "create_model":
      return await handleCreateModel(request.params.arguments as any);

    case "export_fbx":
      return await handleExportFBX(request.params.arguments as any);
```

```
    default:
      throw new Error(`Unknown tool: ${request.params.name}`);
  }
});

// ============================================
// INICIAR SERVIDOR
// ============================================

async function main() {
  // Validar instalación de Blender
  const isValid = await cli.validateInstallation();
  if (!isValid) {
    console.error(
      "[Blender MCP] ERROR: Blender not found or not working at",
      config.blenderExecutable
    );
    process.exit(1);
  }

  console.error(
    "[Blender MCP] Blender found, server ready with config:",
    JSON.stringify(config, null, 2)
  );

  await server.connect(process.stdin, process.stdout);
}

main().catch((error) => {
  console.error("Fatal error:", error);
  process.exit(1);
});
```

ETAPA 2: Herramientas Básicas de Modelado (Fase I)

Incluidas en 1.7:

- CreateModel (humanoid, cube, sphere)
- ExportFBX

ETAPA 3: Rigging (Fase II)

Script Python en `python/add_rig.py`:

```
import bpy
import sys

# Argumentos desde MCP
obj_name = sys.argv[-1]  # Último argumento
```

```
obj = bpy.data.objects.get(obj_name)
if not obj:
    print(f"Object not found: {obj_name}")
    sys.exit(1)

# Crear armadura
bpy.ops.object.armature_add()
armature = bpy.context.active_object
armature.name = f"{obj_name}_Rig"

print(f"Rig added to {obj_name}")
```

ETAPA 4: Animación (Fase III)

Script Python en `python/create_animation.py`:

```
import bpy

# (Truncado por brevedad - incluir lógica de keyframes)
```

ETAPA 5: Exportación (Fase IV)

Cubierta en ExportFBX (1.7).

Checklist

Etapa 1 ☑

- ☐ Setup Node.js/TypeScript
- ☐ Validar instalación Blender
- ☐ Crear tipos y configuración
- ☐ BlenderCLI básico funciona

Etapa 2

- ☐ CreateModel (humanoid, cube, sphere) funciona
- ☐ ExportFBX funciona

Etapa 3

- ☐ AddRig funciona

Etapa 4

- ☐ CreateAnimation funciona

Etapa 5

- ☐ BatchExport funciona

Próximas Etapas

Una vez completes Blender MCP, sigue a:

- **05_Setup.md** - Configuración completa en Claude Code